LLAP - A simple elegant way to talk to objetcs

Category: LLAP - Lightweight Local Automation Protocol
Last Updated on Friday, 21 June 2013 13:54

This document is no longer maintained.

Please refer to the new LLAP Reference Guide

LLAP - A WAY TO TALK TO OBJECTS

What is LLAP?

LLAP (lightweight local automation protocol) is a simple method for taking to tiny computerised "devices". These devices can monitor or control the everyday objects around us. These devices are connected to a star shaped network (a hub or master being in the centre) multiple networks can be joined easily. It utilises clear text or ASCII (usually via serial communications) so that it can be used easily across many different media types including plain old wires, 2 way radio, CAN, Ethernet and Infra Red. The "language" these devices talk is called LLAP for short.

LLAP has several aims

1. To be human readable (as far as possible, this means it is easy to use and simple to learn)
2. To be compressible to fit into an 8 byte message (means it can be transmitted over existing types of network CAN, LIN, ANT etc)
3. To be run on very basic hardware (essentially anything with a serial port)
4. To be extremely low cost

How does it work?

The protocol is based upon a 12 character message. these messages always start with the letter 'a'. The next two characters identify the device ID (address) and the remaining nine characters form the data being exchanged. If the data is less than nine characters then it is right-padded with '-' (dash) characters. It is assumed that the "communication transceiver" provides for message integrity (via CRC etc.). Messages that get corrupted are removed (intentionally lost) by the transceivers. The incomplete exchange indicates the message can then be "retried" if desired by a hub.

There are two main types of device:

1. Devices that are polled for data at regular intervals by the hub (similar to MODBUS and the main stay of operation). These devices are normally permanently powered or are continually charged e.g. solar or energy harvesting.

2. Devices that send data when they have something to send. This further splits in two common types

    a. Devices that sleep most of the time and occasionally wake up and transmit readings. These are usually cyclic and repeat their behaviour. (they are often battery powered, the sleeping is to conserve energy)
    b. Devices that need to send an alert, e.g. a doorbell, smoke sensor, overflow detection etc. Their behaviour is to report at the moment the action or detection occurred. (these can be powered by any means)

The message structure

As mentioned earlier the message is 12 characters long and in 3 distinct sections. To illustrate the 3 separate parts to the message, see this example:

aXXDDDDDDDDD

1. "a" is lower case and shows the start of the message
2. XX is the device identifier (address A-Z & A-Z)
3. DDDDDDDDDD is the data being exchanged.

Note: Only the first "a" character is lowercase, the remaining message always uses uppercase.

Device startup

A new "out of the box" device is pre-assigned the generic ID of **--** (dash dash). When it is powered up it sends
a--STARTED—
if a hub hears this, it replies with
a--ACK------

If no response is heard then a further 5 attempts (retries) are made, again in the same format.
a--STARTED—
a--STARTED—
a--STARTED—
a--STARTED—
a--STARTED—
The device now assumes normal operation on the ID of –

Address allocation

Assuming the hub did hear the device it begins by allocating it a unique address. The hub will send out

a—CHDEVIDXX

where XX represents the ID to allocate to the device. The device responds to this by echoing the command

a—CHDEVIDXX

The hub then requests the device to restart thus applying the new ID

a--REBOOT---

The device then resets and starts up by announcing

aXXSTARTED--

This is repeated at 193mS intervals for an additional 5 times unless the hub replies with

aXXACK------

This startup message now happens every time that the device is reset or powered up and acts to inform the hub that the device is available.

Generic message exchange

Commands from the hub, are acknowledged by the device by one of two methods. Either repeating the command (simple acknowledgment of receipt) or replying with the data requested. The response "should" be sent back within 0.1 second (100ms). If no receipt message is received then further requests can be sent. The number of retries is normally 5, however this can be changed within the hub or the device.

Example:

aXXREBOOT---

The device sends back a similar message to acknowledge receipt.

aXXREBOOT---

Standard LLAP commands

CHDEVID – Change device ID

aXXCHDEVIDDD
Request that the device change its device ID to DD. The change will only take place when the device is rebooted or powered on/off.

PANID – Change PANID

aXXPANIDFFFF
Requests that the device change its PANID to FFFF(the id of the channel the device listens to). PANID is a four character hexadecimal number on XRF or XBee wireless networks. All devices within the same PANID can communicate with each other. The change will only take place when the device is reset or powered on/off.

REBOOT – Force a restart

aXXREBOOT---
Request that the device reset itself.

APVER – Request the LLAP version

aXXAPVER----
Requests the supported LLAP version number – response from the device is aXXAPVER9.99 where 9.99 is the version number. The current version at present is 1.0

DEVTYPE – Request the device type

aXXDEVTYPE--
Request the device type. Response is a nine character device type e.g. aXXU00000001 Device types beginning with U are reserved for users to assign to their own prototype devices. All other device types will be kept on a register by CISECO Plc., that can be used to update hubs on a regular basis. In APVER 1.1 this will change to a eight character field e.g. aXXU0000001-

DEVNAME – Request the manufacturer device name

aXXDEVNAME--
Request the device name. Response is a nine character "friendly" name.

HELLO – Request the device to acknowledge it's there.

aXXHELLO----
Request the device to confirm it is receiving, response is
aXXHELLO----

SER – Request the manufacturer serial number

aXXSER------
Request the device serial number, response is a 6 character serial number, e.g. aXXSER000001

FVER – Request the manufacturer firmware version

aXXFVER-----
Request the device firmware version. Response is aXXFVER9.99- e.g. aXXFVER0.32-

RETRIES – Set the amount of messages to retry to get an ACK from the hub

aXXRETRIESRR
Set the amount of retries that uninitiated "sends" should try before giving up, default is 5 (number can be 00-99) this is set by RR

SLEEP – Requests the device go into low power mode (only applies to sleeping devices)

aXXSLEEP999P
Request that the device sleep for 999 "periods", the response echos the command, the device will then send the notification aXXSLEEPING- when going to sleep, and the announcement aXXAWAKE---- when the device reawakens. Periods can be S(seconds), M(minutes), H(hours), D(days)

aXXSLEEP---- (only applies to interrupt sleeping devices) . Sleep until an interrupt is received. Only the Button firmware behaves like this at present.
Every (default 10) times it will send aXXAWAKE---- aXXBATTv.vv- wait for 100mS incase there is a command for the unit and then send aXXSLEEPING-. This is configurable by using aXXWAKECnnn- whenre nnn is the count of transmits before it will wake and send the battery reading.

INTVL – Sets the sleep interval between "activities" (only applies to cyclic sleeping devices)

XXINTVL999P
Set the cyclic sleep interval to 999 "periods"
Periods can be S(seconds), M(minutes), H(hours), D(days)

CYCLE – Start cyclic mode (only applies to cyclic sleeping devices)

aXXCYCLE---
Request that the device starts a cyclic sleep – the device should sleep for the sleep interval, awaken and send any reading. The device wiull then go back to sleep. Every (default 10) times it will send aXXAWAKE---- aXXBATTv.vv- wait for 100mS incase there is a command for the unit and then send aXXSLEEPING-. This is configurable by using aXXWAKECnnn- whenre nnn is the count of transmits before it will wake and send the battery reading.

BATT - Request battery level in Volts

aXXBATT----- will reply aXXBATT3.43- (3.43V)

Device related "announcements" (messages that are not "requested")

STARTED

aXXSTARTED-- This is to notify the hub that a device has just started. The hub can then add the device back into the list of active devices.

ERROR

aXXERRORnnnn - This is to notify that something unexpected happened at the device and that should be investigated. The nnnn can be used by the manufacturer to denote what type of error occurred.

SLEEPING- Device is going to sleep (only applies to devices that support sleep)

aXXSLEEPING

AWAKE – Device has re awoken (only applies to devices that support sleep)

aXXAWAKE - Device is now awake, <Possible addition: if the device is a cyclic sleep device then the device should now remain awake for at least 100mS listening for hub requests. Such a device does not need to announce that it is awake at each cycle, but is expected to do so at least every 5 minutes.

BATTLOW - Device battery is low (only applies to battery powered devices)

aXXBATTLOW

< Prev Next >